

ADL - Challenge Report

Bumblebees

İsmail Emre Gümüş, Neslihan Pelin Metin, Merve Rana Kızıl, Samet Aydın

emregms017, neslihanpelinmetin, merveranakizil, sametaydin1

274803, 302544, 271365, 269770

May 25, 2026

1 Introduction

This project addresses the ADL anomaly detection challenge, where the goal is to produce accurate *pixel-level anomaly maps* for multi-view industrial images. Each test image must be assigned a two-dimensional anomaly score map, which is finally encoded in the required q8rle submission format.

Our final solution evolved from a purely feature-based normality baseline to a strong ensemble of supervised segmentation models and lightweight normality detectors. The main objective was not only to detect whether an image was anomalous, but also to localise the defective regions as precisely as possible. The final public best submission reached **0.8803**, while our best private leaderboard result was **0.9000**, obtained by the Run41 specialist blend.

2 Problem Analysis

The dataset contains eight object classes, denoted as `class_01` to `class_08`. For each class, training data includes a large number of clean images and a much smaller number of real anomalous images with ground-truth masks. The test set contains 5910 images in total.

The main difficulty of the challenge is the limited number of real anomaly examples. This makes direct supervised learning prone to overfitting. Moreover, different classes show different visual structures and anomaly patterns, so a single global model or a single global ensemble weight is not always optimal.

For this reason, the final pipeline combines three ideas:

- synthetic anomaly generation to increase training diversity;
- multiple segmentation architectures to obtain complementary predictions;
- class-specific blending to exploit different model strengths per class.

3 Method

Our method is based on a progressive ensemble pipeline. Early runs used unsupervised normality models, while later runs focused on supervised semantic segmentation trained with real and synthetic anomaly masks.

3.1 Synthetic anomaly segmentation

The most important improvement came from training segmentation models on a mixture of clean images, real anomaly masks, and synthetic anomalies. Given an input image x , each segmentation model predicts a dense anomaly probability map:

$$\hat{M} = f_{\theta}(x), \quad (1)$$

where $\hat{M}_{ij}$ represents the anomaly score of pixel (i, j) .

The supervised models were trained with an AdamW optimizer, cosine learning-rate scheduling, and ImageNet-pretrained encoders. We used architectures from `segmentation_models.pytorch`,

mainly UNet++, DeepLabV3+, and FPN. The synthetic anomalies acted as additional positive samples and were essential for reducing the gap between the low number of real masks and the much larger test distribution.

3.2 Normality-based anomaly maps

In addition to supervised segmentation, we used feature-space normality detectors. PatchCore and PaDiM extract intermediate CNN features from good training images only. At inference time, pixels or patches far from the training-good feature distribution receive higher anomaly scores.

This family of models is complementary to segmentation networks because it does not rely on synthetic mask generation. Instead, it models normal appearance and highlights deviations from it. This signal was reintroduced in Run42 as a conservative final blend.

3.3 Ensembling

The final prediction is a weighted average of multiple anomaly maps:

$$M_{\text{final}} = \sum_{k=1}^K w_k M_k, \quad \sum_{k=1}^K w_k = 1. \quad (2)$$

We first used global weights, then moved to class-specific micro-blending. This was important because some models improved only a subset of classes. For example, Run41 was a specialist model trained only on `class_01-class_05`, and its predictions were blended only for those classes while keeping the previous best unchanged for `class_06-class_08`.

4 Experiments

Table 1 summarises the direct winning path from the initial baseline to the final submissions.

4.1 Main runs

The initial Run01 baseline used PatchCore, PaDiM, and RevDistill with equal-weight averaging. Although it provided a useful first submission, its score was limited because it did not learn from the available anomaly masks.

Run12 introduced the first supervised segmentation model. It used UNet++ with an EfficientNet-B3 encoder at 320 px resolution and trained on clean images, real anomalies, and synthetic anomaly masks. This produced the largest single improvement, raising the public score from 0.4785 to 0.8316 after blending.

Run18 replaced the segmentation architecture with DeepLabV3+ and a ResNet50 encoder. This added a stronger architectural signal and increased the public score to 0.8638. Run20 then introduced an FPN with EfficientNet-B4, improving multi-scale feature aggregation and reaching 0.8694.

Run28B used an FPN with a ResNet34 encoder as a stable independent signal. Although simpler than some attempted alternatives, it had different failure modes and improved the ensemble to 0.8787. Run31 then performed class-specific micro-blending using mostly Run28B, with small contributions from Run20 and Run18.

Later, Run40 added a UNet++ EfficientNet-B4 model at 352 px resolution. It was blended conservatively with the current best using a 95%/5% ratio, reaching 0.8800. Run41 trained a lightweight FPN ResNet18 specialist for `class_01-class_05`. Its best blend used 96% of the previous best and 4% of the specialist on those classes only, producing the best private score of 0.9000. Finally, Run42 reintroduced PatchCore and PaDiM normality maps and obtained the best public score of 0.8803.

5 Results

The largest gain came from moving from unsupervised normality detection to supervised segmentation with synthetic anomalies. This single change contributed more than +0.35 public score. After that, improvements became smaller but consistent, mostly coming from architectural diversity and careful blending.

The final results show that no single model family was sufficient by itself. UNet++, DeepLabV3+, FPN, class-specialist training, and normality-based detectors each contributed different information. The best public and private submissions were not exactly the same:

- **Best public score:** Run42 with 0.8803;
- **Best private score:** Run41 with 0.9000.

Table 1: Score progression of the main experimental path. Best scores are highlighted in bold.

Run	Method	Public	Δ Public	Private	Δ Private
Run01	PatchCore + PaDiM + RevDistill baseline	0.4785	–	0.4768	–
Run12	UNet++ synthetic segmentation + blend	0.8316	+0.3531	0.8494	+0.3726
Run18	DeepLabV3+ ResNet50 blend	0.8638	+0.0322	0.8815	+0.0321
Run20	FPN EfficientNet-B4 blend	0.8694	+0.0056	0.8907	+0.0092
Run28B	FPN ResNet34 independent signal	0.8787	+0.0093	0.8984	+0.0077
Run31	Class-specific micro blend	0.8795	+0.0008	0.8988	+0.0004
Run40	UNet++ EfficientNet-B4 at 352 px	0.8800	+0.0005	0.8996	+0.0008
Run41	FPN ResNet18 class_01–05 specialist	0.8802	+0.0002	0.9000	+0.0004
Run42	PatchCore + PaDiM normality ensemble	0.8803	+0.0001	0.8967	-0.0033

This difference suggests that the very small normality-based correction in Run42 slightly improved the public split, but did not generalise as well to the private split. Therefore, Run41 can be considered the most robust final model according to the private leaderboard.

To qualitatively inspect the spatial behaviour of the final model, we decoded the anomaly maps from the original Run41 submission CSV and overlaid the detected regions on the corresponding test images. Figure 1 shows representative examples from all eight classes. Since test-set ground-truth masks are not available, these visualisations are intended only as qualitative examples.

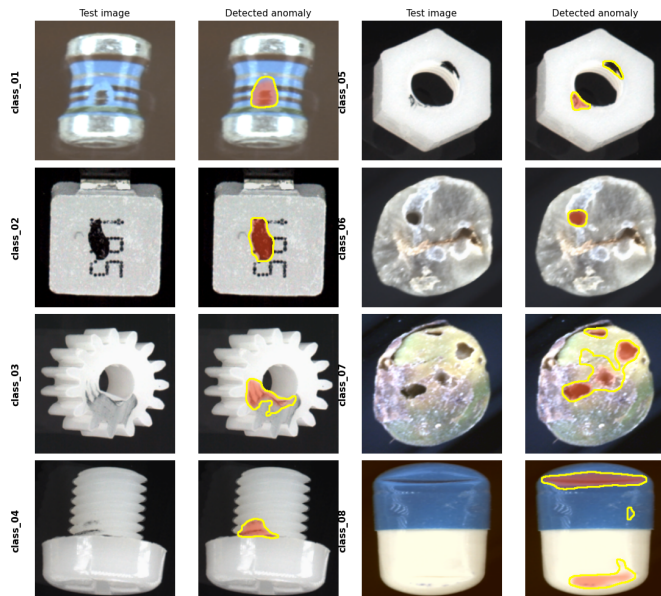


Figure 1: Qualitative detected anomaly examples from the test set.

6 Discussion

The main strength of the final approach is model diversity. Supervised segmentation models are strong when synthetic anomalies match the test distribution, while PatchCore and PaDiM provide an independent normality signal that is less dependent on synthetic generation choices.

However, the method also has limitations. First, the final improvements after Run28B were very small, which means many blends were close to the noise level of leaderboard feedback. Second, class-specific blending was tuned using leaderboard observations, so it may overfit the public split. Third, synthetic anomaly generation is useful but imperfect: if synthetic defects do not resemble real defects, the segmentation model may learn artifacts that do not transfer.

7 Conclusions

In this project, we developed a complete anomaly segmentation pipeline for the ADL challenge. Starting from a PatchCore/PaDiM/RevDistill baseline, we progressively improved the solution using supervised segmentation, synthetic anomaly generation, architectural diversity, class-specific blending, and final normality-based correction.

Our final public best score was **0.8803** with Run42, while the best private score was **0.9000** with Run41. The most important lesson is that the strongest solution came from combining complementary model families rather than relying on a single architecture. Future work could explore stronger synthetic anomaly realism, better validation splits for class-specific blending, and more systematic calibration of anomaly maps before ensembling.